AI-ASSISTED CODING

ASSIGNMENT-9.5

NAME:V.SIDDARTHA
HALL:2303A52062
BATCH:45

Problem 1: String Utilities Function
Consider the following Python function:
def reverse_string(text):
return text[::-1]
Task:
1. Write documentation in:
o (a) Docstring
o (b) Inline comments
o (c) Google-style documentation
2. Compare the three documentation styles.
3. Recommend the most suitable style for a utility-based string library.

PROMPT:
Document reverse_string using docstring, inline comments, and Google style, compare them, and recommend one for utility libraries."""

CODE:

```
def reverse_string_docstring(text):
    """Return the reversed version of the input string."""
    return text[::-1]

def reverse_string_inline(text):
    # Slice the string from end to start with step -1.
    return text[::-1]

def reverse_string_google(text):
    """Reverse a string.

    Args:
        text (str): Input text to reverse.

    Returns:
        str: Reversed text.
    """
```

```
    return text[::-1]

comparison = {
    "Docstring": "Short and simple; good for quick utilities.",
    "Inline comments": "Useful only for non-obvious logic; too many can add noise.",
    "Google style": "Most structured and tool-friendly; best for teams and generated docs.",
}

recommendation = "Google style is most suitable for a utility-based string library."

if __name__ == "__main__":
    sample = "Utility123"
    print("Task 1 Output")
    print("Reverse (Docstring):", reverse_string_docstring(sample))
    print("Reverse (Inline):", reverse_string_inline(sample))
    print("Reverse (Google):", reverse_string_google(sample))
    print("Comparison:")
    for style, note in comparison.items():
        print(f"- {style}: {note}")
    print("Recommendation:", recommendation)
```

OUTPUT:

```
PROBLEMS    DEBUG CONSOLE    TERMINAL    PORTS    POSTMAN CONSOLE

PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\Local\Programs\Python\Python314\pytho
n.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Assisted coding/.github/9.5 ASSIGN.PY/TASK1.PY"
Task 1 Output
Reverse (Docstring): 321ytilitU
Reverse (Inline): 321ytilitU
Reverse (Google): 321ytilitU
Comparison:
- Docstring: Short and simple; good for quick utilities.
- Inline comments: Useful only for non-obvious logic; too many can add noise.
- Google style: Most structured and tool-friendly; best for teams and generated docs.
Recommendation: Google style is most suitable for a utility-based string library.
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>

powershell
Python
Python
```

Problem 2: Password Strength Checker

Consider the function:

```
def check_strength(password):
return len(password) >= 8
```

Task:

1. Document the function using docstring, inline comments, and Google style.

2. Compare documentation styles for security-related code.

3. Recommend the most appropriate style.

PROMPT:
Document check_strength with docstring, inline comments, and Google style, compare styles for security code, and recommend the best style."""

CODE:

```
def check_strength_docstring(password):
    """Return True if password length is at least 8 characters."""
    return len(password) >= 8

def check_strength_inline(password):
    # Minimum length baseline for basic password strength.
    return len(password) >= 8

def check_strength_google(password):
    """Check whether a password meets basic minimum length.

    Args:
        password (str): Password value to validate.

    Returns:
        bool: True when length is 8 or more, otherwise False.
    """
    return len(password) >= 8

comparison = {
    "Docstring": "Quick readability, but limited detail for security context.",
    "Inline comments": "Can clarify checks, but scattered and easy to miss.",
    "Google style": "Clear contract (Args/Returns), easy for audits and docs.",
}

recommendation = "Google style is most appropriate for security-related code due to clarity and auditability."

if __name__ == "__main__":
    print("Task 2 Output")
    print("'abc12345' ->", check_strength_google("abc12345"))
    print("'pass' ->", check_strength_google("pass"))
    print("Comparison:")
    for style, note in comparison.items():
        print(f"- {style}: {note}")
    print("Recommendation:", recommendation)
```

OUTPUT:

```
PROBLEMS    DEBUG CONSOLE    TERMINAL    PORTS    POSTMAN CONSOLE

PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\Local\Programs\Python\Python314\pytho
n.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Assisted coding/.github/9.5 ASSIGN.PY/TASK2.PY"
Task 2 Output
'abc12345' -> True
'pass' -> False
Comparison:
- Docstring: Quick readability, but limited detail for security context.
- Inline comments: Can clarify checks, but scattered and easy to miss.
- Google style: Clear contract (Args/Returns), easy for audits and docs.
Recommendation: Google style is most appropriate for security-related code due to clarity and auditability.
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```

Problem 3: Math Utilities Module

Task:

1. Create a module math_utils.py with functions:

o square(n)

o cube(n)

o factorial(n)

2. Generate docstrings automatically using AI tools.

3. Export documentation as an HTML file.

PROMPT:

Create math_utils.py with square/cube/factorial, include AI-generated docstrings, and export module documentation as HTML."""

CODE:

```
import os
import pydoc

import math_utils

def export_math_docs_html():
    html = pydoc.HTMLDoc().docmodule(math_utils)
    out_path = os.path.join(os.path.dirname(__file__), "math_utils_docs.html")
    with open(out_path, "w", encoding="utf-8") as f:
        f.write(html)
    return out_path

if __name__ == "__main__":
    print("Task 3 Output")
    print("square(5) =", math_utils.square(5))
    print("cube(3) =", math_utils.cube(3))
    print("factorial(5) =", math_utils.factorial(5))
    html_file = export_math_docs_html()
    print("Documentation exported to:", html_file)
```

DOCUMENTATION:


```
<table class="heading">
<tr class="heading-text decor">
<td class="title"> <br><strong class="title">math_utils</strong></td>
<td class="extra"><a href=".">index</a><br><a
href="file:c%3A%5Cusers%5Csidda%5Conedrive%5Cdocuments%5Cdesktop%5Cai%20-
%20assisted%20coding%5C.github%5C9.5%20assign.py%5Cmath_utils.py">c:\users\sidda
\onedrive\documents\desktop\ai - assisted coding\.github\9.5
assign.py\math_utils.py</a></td></tr></table>
   <p><span class="code">Math utility functions with AI-
generated docstrings.</span></p>
<p>
<table class="section">
<tr class="decor functions-decor heading-text">
<td class="section-title" colspan=3> <br><strong
class="bigsection">Functions</strong></td></tr>

<tr><td class="decor functions-decor"><span
class="code">      </span></td><td> </td>
<td class="singlecolumn"><dl><dt><a name="-
cube"><strong>cube</strong></a>(n)</dt><dd><span
class="code">Return the cube of n.<br>
 <br>
Args:<br>
    n (int | float): Numeric value
.<br>
 <br>
Returns:<br>
    int | float: n raised to 
power 3.</span></dd></dl>
 <dl><dt><a name="-factorial"><strong>factorial</strong></a>(n)</dt><dd><span
class="code">Return factorial of a non-
negative integer n.<br>
 <br>
Args:<br>
    n (int): Non-negative integer.<br>
 <br>
Returns:<br>
```

    int: Product of all integers from 1 to n.

Raises:
    ValueError: If n is negative or not an integer.</dd></dl>
<dl><dt><a name="-square">square</a>(n)</dt><dd>Return the square of n.

Args:
    n (int | float): Numeric value.

Returns:
    int | float: n multiplied by itself.</dd></dl>
</td></tr></table>

index
math_utils c:\users\sidda\onedrive\documents\desktop\ai - assisted coding\.github\9.5 assign.py\math_utils.py

Math utility functions with AI-generated docstrings.

Functions

cube(n)
Return the cube of n.

Args:
n (int | float): Numeric value.

Returns:
int | float: n raised to power 3.

factorial(n)
Return factorial of a non-negative integer n.

Args:
n (int): Non-negative integer.

Returns:
int: Product of all integers from 1 to n.

Raises:
ValueError: If n is negative or not an integer.

square(n)
Return the square of n.

Args:
n (int | float): Numeric value.

Returns:
int | float: n multiplied by itself.

```
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\Local\Programs\Python\Python314\pytho
n.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Assisted coding/.github/9.5 ASSIGN.PY/TASK3.PY"
Task 3 Output
square(5) = 25
cube(3) = 27
factorial(5) = 120
Documentation exported to: c:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding\.github\9.5 ASSIGN.PY\math_
tml
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```

Live Reload is not
Source: Live Server

Problem 4: Attendance Management Module

Task:

1. Create a module attendance.py with functions:
o mark_present(student)
o mark_absent(student)
o get_attendance(student)
2. Add proper docstrings.
3. Generate and view documentation in terminal and browse

PROMPT:
Build attendance.py with docstrings, then generate and view documentation in terminal and browser."""
CODE:

```
import os
import pydoc
import webbrowser

import attendance

def export_attendance_docs_html():
    html = pydoc.HTMLDoc().docmodule(attendance)
    out_path = os.path.join(os.path.dirname(__file__), "attendance_docs.html")
    with open(out_path, "w", encoding="utf-8") as f:
        f.write(html)
    return out_path

if __name__ == "__main__":
    attendance.mark_present("Alice")
    attendance.mark_absent("Bob")

    print("Task 4 Output")
    print("Alice ->", attendance.get_attendance("Alice"))
    print("Bob ->", attendance.get_attendance("Bob"))
    print("Charlie ->", attendance.get_attendance("Charlie"))

    print("\nTerminal Documentation Preview:")
    print(pydoc.render_doc(attendance, title="%s"))

    html_file = export_attendance_docs_html()
    print("HTML documentation exported to:", html_file)

    try:
        webbrowser.open("file://" + html_file)
```

```
        print("Browser opened for attendance documentation.")
    except Exception:
        print("Could not open browser automatically; open the HTML file manually.")
```

DOCUMENTATION:


```
<table class="heading">
<tr class="heading-text decor">
<td class="title"> <br><strong class="title">attendance</strong></td>
<td class="extra"><a href=".">index</a><br><a
href="file:c%3A%5Cusers%5Csidda%5Conedrive%5Cdocuments%5Cdesktop%5Cai%20-
%20assisted%20coding%5C.github%5C9.5%20assign.py%5Cattendance.py">c:\users\sidda
\onedrive\documents\desktop\ai - assisted coding\.github\9.5
assign.py\attendance.py</a></td></tr></table>
    <p><span
class="code">Attendance management module with simple&nb
sp;in-memory storage.</span></p>
<p>
<table class="section">
<tr class="decor functions-decor heading-text">
<td class="section-title" colspan=3> <br><strong
class="bigsection">Functions</strong></td></tr>

<tr><td class="decor functions-decor"><span
class="code">      </span></td><td> </td>
<td class="singlecolumn"><dl><dt><a name="-
get_attendance"><strong>get_attendance</strong></a>(student)</dt><dd><span
class="code">Get attendance status of a student.<br>
 <br>
Args:<br>
    student (str): Student name or&nb
sp;ID.<br>
 <br>
Returns:<br>
    str: 'Present', 'Absent', or 'Not&nbs
p;Marked'.</span></dd></dl>
 <dl><dt><a name="-
mark_absent"><strong>mark_absent</strong></a>(student)</dt><dd><span
class="code">Mark a student as Absent.<br>
 <br>
Args:<br>
```

    student (str): Student name or ID.</dd></dl>
<dl><dt><a name="-mark_present">mark_present</a>(student)</dt><dd>Mark a student as Present.

Args:
    student (str): Student name or ID.</dd></dl>
</td></tr></table>

```
n.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Assisted coding/.github/9.5 ASSIGN.PY/TASK4.PY"
Task 4 Output
Alice -> Present
Bob -> Absent
Charlie -> Not Marked

Terminal Documentation Preview:
module attendance

NAME
    attendance - Attendance management module with simple in-memory storage.

FUNCTIONS
    get_attendance(student)
        Get attendance status of a student.

        Args:
            student (str): Student name or ID.

        Returns:
            str: 'Present', 'Absent', or 'Not Marked'.

    mark_absent(student)
        Mark a student as Absent.

        Args:
            student (str): Student name or ID.

    mark_present(student)
        Mark a student as Present.

        Args:
            student (str): Student name or ID.

FILE
    c:\users\sidda\onedrive\documents\desktop\ai - assisted coding\.github\9.5 assign.py\attendance.py


HTML documentation exported to: c:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding\.github\9.5 ASSIGN.PY\attendance_d
```

index
attendance c:\users\sidda\onedrive\documents\desktop\ai - assisted coding\.github\9.5 assign.py\attendance.py

Attendance management module with simple in-memory storage.

Functions

get_attendance(student)
Get attendance status of a student.

Args:
student (str): Student name or ID.

Returns:
str: 'Present', 'Absent', or 'Not Marked'.

mark_absent(student)
Mark a student as Absent.

Args:
student (str): Student name or ID.

mark_present(student)
Mark a student as Present.

Args:
student (str): Student name or ID.

Problem 5: File Handling Function

Consider the function:

```
def read_file(filename):
with open(filename, 'r') as f:
return f.read()
```

Task:

1. Write documentation using all three formats.
2. Identify which style best explains exception handling.
3. Justify your recommendation.

PROMPT:

Document read_file in docstring, inline comments, and Google style, compare exception-handling clarity, and justify the best style."""

CODE:

```
def read_file_docstring(filename):
    """Read and return full file content.

    Returns empty text if file cannot be read.
    """
    try:
        with open(filename, "r", encoding="utf-8") as f:
            return f.read()
    except OSError:
        return ""
```

```
def read_file_inline(filename):
    try:
        # Open file in text mode and read complete content.
        with open(filename, "r", encoding="utf-8") as f:
            return f.read()
    except OSError:
        # Handle file-not-found/permission issues gracefully.
        return ""

def read_file_google(filename):
    """Read and return full file content safely.

    Args:
        filename (str): Path to input text file.

    Returns:
        str: File content, or empty string on read failure.

    Raises:
        None: Exceptions are handled internally.
    """
    try:
        with open(filename, "r", encoding="utf-8") as f:
            return f.read()
    except OSError:
        return ""

analysis = {
    "Best for exception handling": "Google style",
    "Why": "It explicitly documents behavior, inputs, outputs, and exception strategy in one
place.",
}

if __name__ == "__main__":
    demo_file = "task5_demo.txt"
    with open(demo_file, "w", encoding="utf-8") as f:
        f.write("Hello File Handling")

    print("Task 5 Output")
    print("Docstring style read:", read_file_docstring(demo_file))
```

```
print("Inline style read:", read_file_inline(demo_file))
print("Google style read:", read_file_google(demo_file))
print("Best style:", analysis["Best for exception handling"])
print("Justification:", analysis["Why"])
```

OUTPUT:

PROBLEMS DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\Local\Programs\Python\Python314\pytho
n.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Assisted coding/.github/9.5 ASSIGN.PY/TASK5.PY"
Task 5 Output
Docstring style read: Hello File Handling
Inline style read: Hello File Handling
Google style read: Hello File Handling
Best style: Google style
Justification: It explicitly documents behavior, inputs, outputs, and exception strategy in one place.
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```